

中山大学专业课《程序设计I》

第六节 函数

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季

C Programming

判断素数

7-5 判断一个整数是否为素数

分数 20

[查看题目详情](#) [全屏浏览](#) [切换布局](#)

本题要求编写程序，判断一个给定的整数是否为素数。素数就是只能被1和自身整除的正整数，1不是素数，2是素数。

输入格式:

输入在一行中给出一个需要判断的整数 M ($-2^{31} \leq M \leq 2^{31} - 1$)。

输出格式:

如果 M 是素数，则在一行中输出 **Yes**，否则输出 **No**。如果输入了非正整数，也要输出 **No**。

```
bool is_prime = true;
int m;
scanf("%d", &m);
```

```
for (int i = 2; i < m; i++)
    if (m % i == 0)
    {
        is_prime = false;
        break;
    }
```

```
printf(is_prime && m>1 ? "Yes\n" : "No\n");
```

判断素数

7-5 判断一个整数是否为素数

分数 20

[查看题目详情](#) [全屏浏览](#) [切换布局](#)

本题要求编写程序，判断一个给定的整数是否为素数。素数就是只能被1和自身整除的正整数，1不是素数，2是素数。

输入格式:

输入在一行中给出一个需要判断的整数 M ($-2^{31} \leq M \leq 2^{31} - 1$)。

输出格式:

如果 M 是素数，则在一行中输出 **Yes**，否则输出 **No**。如果输入了非正整数，也要输出 **No**。

```
bool is_prime = true;
int m;
scanf("%d", &m);
```

```
for (int i = 2; i <= m / i; i++)
    if (m % i == 0)
    {
        is_prime = false;
        break;
    }
```

```
printf(is_prime && m>1 ? "Yes\n" : "No\n");
```

哥德巴赫猜想

7-6 验证“哥德巴赫猜想”

分数 20

[查看题目详情](#) [全屏浏览](#) [切换布局](#)

数学领域著名的“哥德巴赫猜想”的大致意思是：任何一个大于2的偶数总能表示为两个素数之和。比如：24=5+19，其中5和19都是素数。本实验的任务是设计一个程序，验证20亿以内的偶数都可以分解成两个素数之和。

输入格式：

输入在一行中给出一个(2, 2 000 000 000]范围内的偶数N。

输出格式：

在一行中按照格式“N = p + q”输出N的素数分解，其中 $p \leq q$ 均为素数。又因为这样的分解不唯一（例如24还可以分解为7+17），要求必须输出所有解中p最小的解。

```
bool is_prime;
int n;
scanf("%d", &n);
```

```
int p = 2;
```

```
for(; p<=n; p++){
```

```
    is_prime = true;
```

```
    for (int i = 2; i <= p / i; i++){
```

```
        if (p % i == 0){
```

```
            is_prime = false;
```

```
            break;
```

```
        }
```

```
    }
```

```
    if(is_prime){
```

```
        for (int i = 2; i <= (n-p) / i; i++){
```

```
            if ((n-p) % i == 0){
```

```
                is_prime = false;
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

```
    if(is_prime) break;
```

```
}
```

```
printf("%d = %d + %d\n", n, p, n-p);
```

```
for (int i = 2; i <= m / i; i++){
```

```
    if (m % i == 0)
```

```
    {
```

```
        is_prime = false;
```

```
        break;
```

```
    }
```

判断素数

用循环

```
bool is_prime;
int n;
scanf("%d", &n);

int p = 2;
for(; p<=n; p++){
    is_prime = true;
    for (int i = 2; i <= p / i; i++){
        if (p % i == 0){
            is_prime = false;
            break;
        }
    }
    if(is_prime){
        for (int i = 2; i <= (n-p) / i; i++){
            if ((n-p) % i == 0){
                is_prime = false;
                break;
            }
        }
    }
    if(is_prime) break;
}

printf("%d = %d + %d\n", n, p, n-p);
```

用循环

```
bool is_prime;
int n;
scanf("%d", &n);

int p = 2;
for(; p<=n; p++){
    is_prime = true;
    for (int i = 2; i <= p / i; i++){
        if (p % i == 0){
            is_prime = false;
            break;
        }
    }
    if(is_prime){
        for (int i = 2; i <= (n-p) / i; i++){
            if ((n-p) % i == 0){
                is_prime = false;
                break;
            }
        }
    }
    if(is_prime) break;
}

printf("%d = %d + %d\n", n, p, n-p);
```

代码重复!

用循环

```
bool is_prime;
int n;
scanf("%d", &n);

int p = 2;
for(; p<=n; p++){
    is_prime = true;
    for (int i = 2; i <= p / i; i++){
        if (p % i == 0){
            is_prime = false;
            break;
        }
    }
    if(is_prime){
        for (int i = 2; i <= (n-p) / i; i++){
            if ((n-p) % i == 0){
                is_prime = false;
                break;
            }
        }
    }
    if(is_prime) break;
}

printf("%d = %d + %d\n", n, p, n-p);
```

用函数

```
int n;
scanf("%d", &n);

for(int p=2; p<=n; p++)
    if(is_prime(p) && is_prime(n-p))
        printf("%d = %d + %d", n, p, n-p);
```

主要内容

1. 函数的概念
2. 函数的定义、调用与声明
3. 递归

函数是什么

函数 (Function) 是实现某功能的独立代码块

我们已经用过了许多函数：

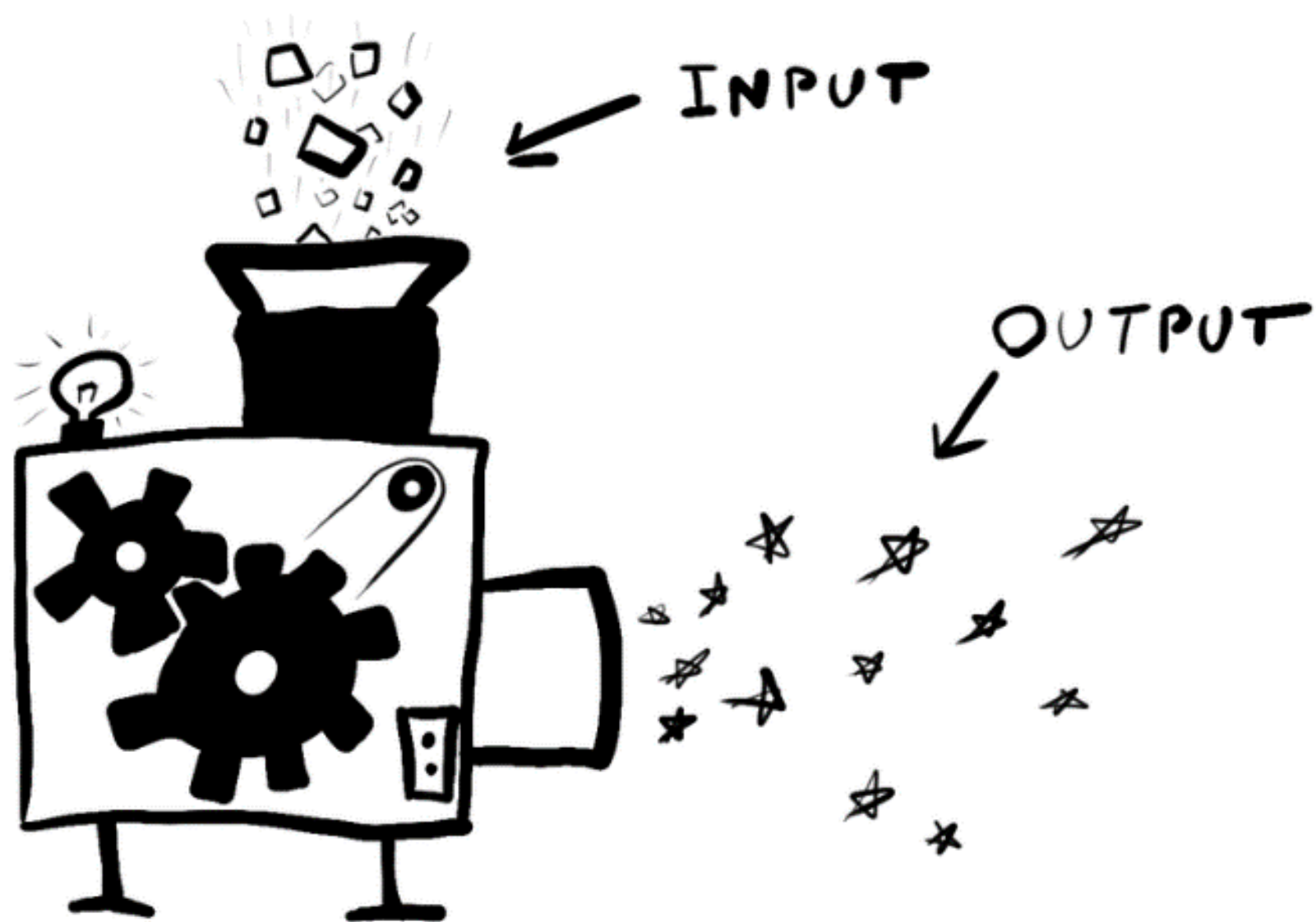
`printf()` `scanf()` `getchar()` `putchar()`

`sqrt()` `pow()` `fabs()`

`strlen()` `isdigit()` `isupper()` ...

函数是什么

- ❑ 函数可理解为一个抽象的“黑盒”：接受“输入”，对输入进行处理并给出“输出”
- ❑ 使用时不需要关注具体的实现细节



输入

输出

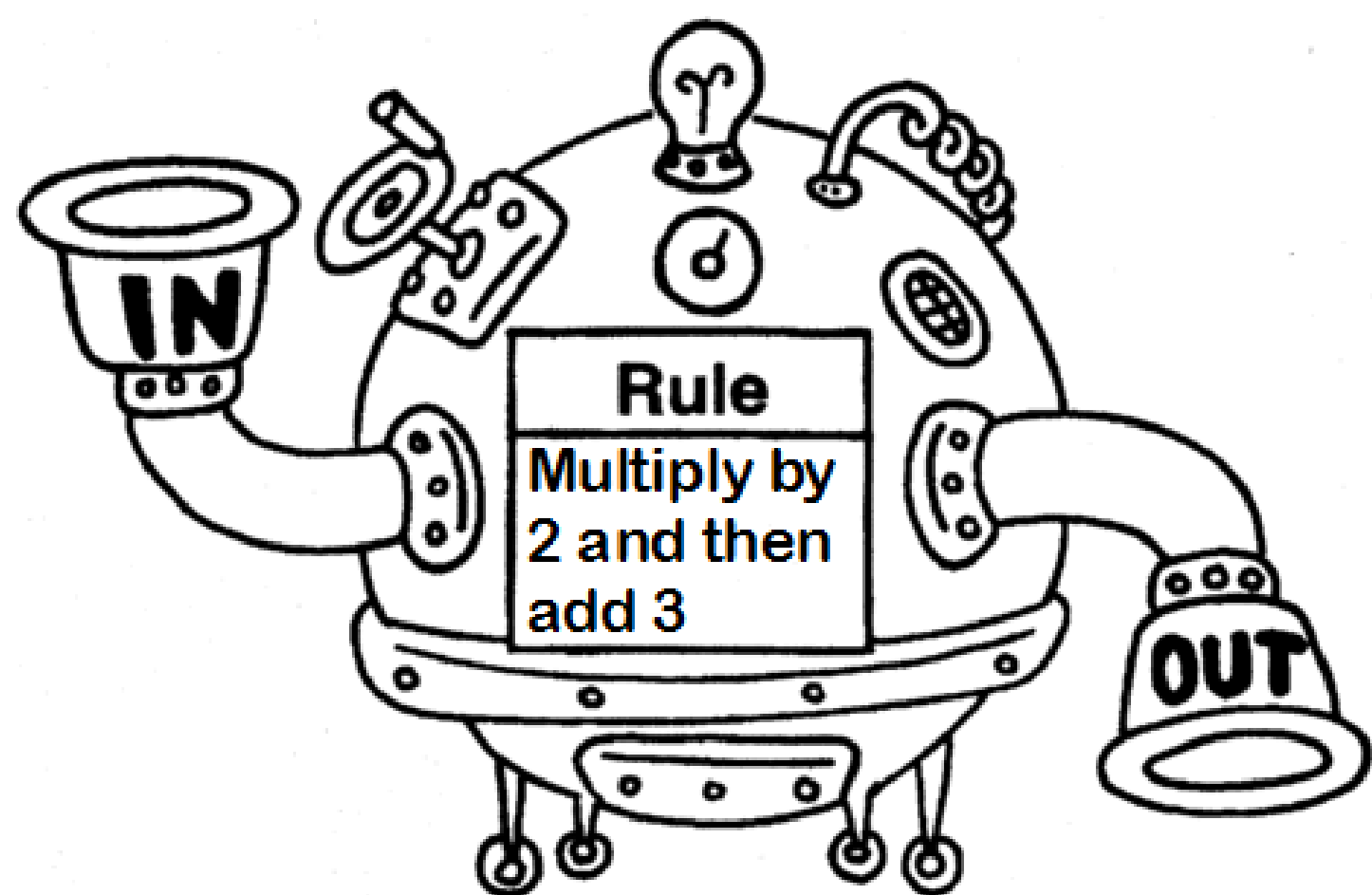
`sqrt(4.0)` - - - - - `2.0`

`printf("A")` - - - - - 打印"A"

`strlen("A")` - - - - - `1`

函数：构建程序的积木块

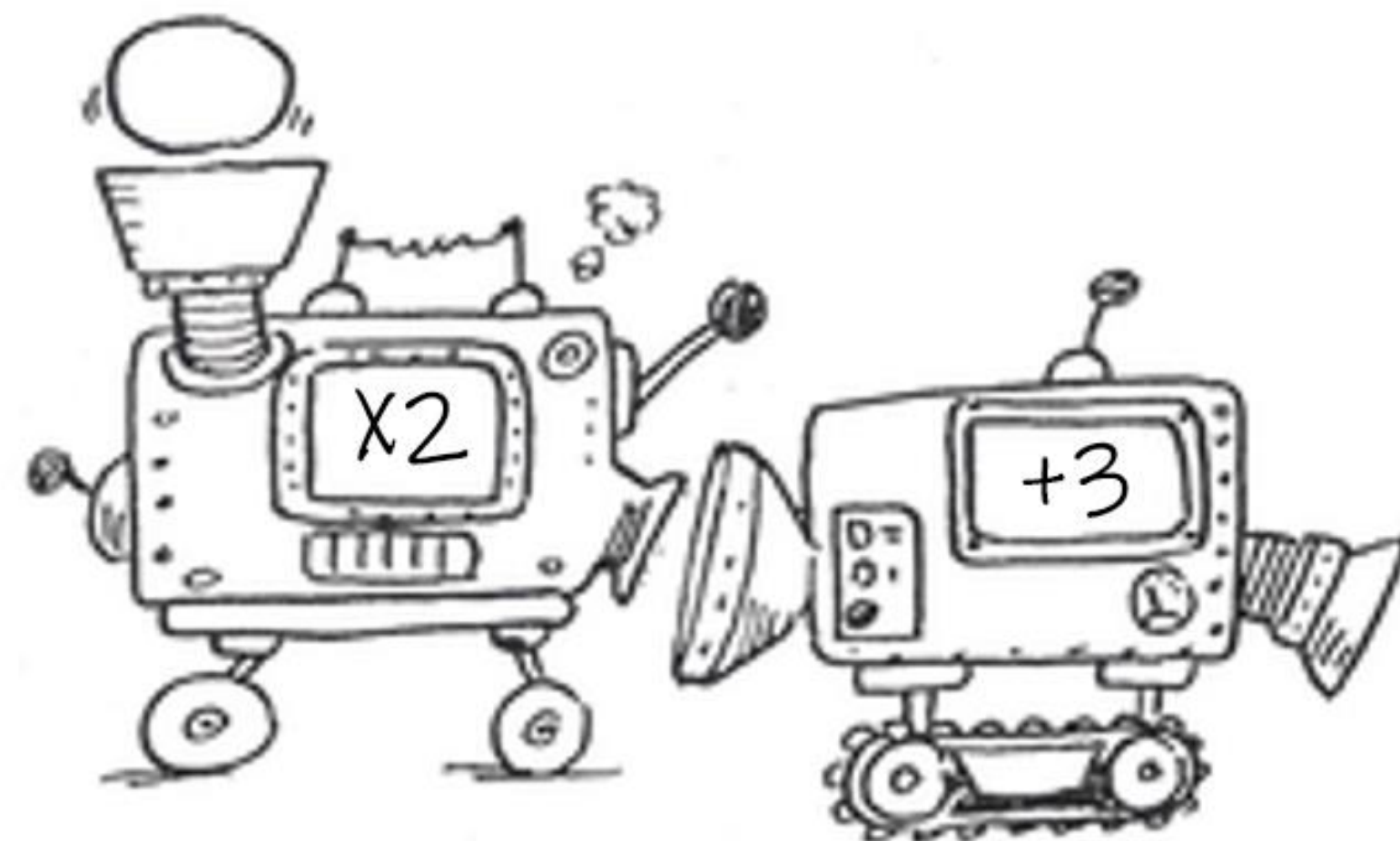
□ 函数能够将一大块代码分解成若干个**模块**



一个大程序

分解

组装



分解成两个小函数

函数：构建程序的积木块

函数能够将代码分解成若干个子模块

```
int n;  
scanf("%d", &n);  
  
for(int p=2; p<=n; p++)  
    if(is_prime(p) && is_prime(n-p))  
        printf("%d = %d + %d", n, p, n-p);
```

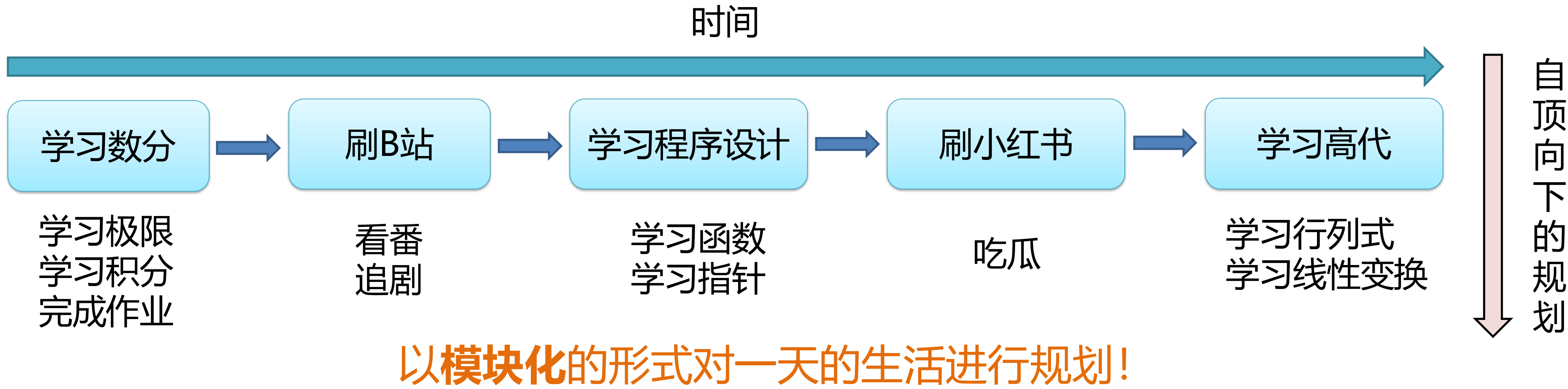
函数：构建程序的积木块

函数能够将代码分解成若干个子模块

```
int n;  
scanf("%d", &n);  
  
for(int p=2; p<=n; p++)  
    if(is_prime(p) && is_prime(n-p))  
        printf("%d = %d + %d", n, p, n-p);
```

函数带来哪些好处

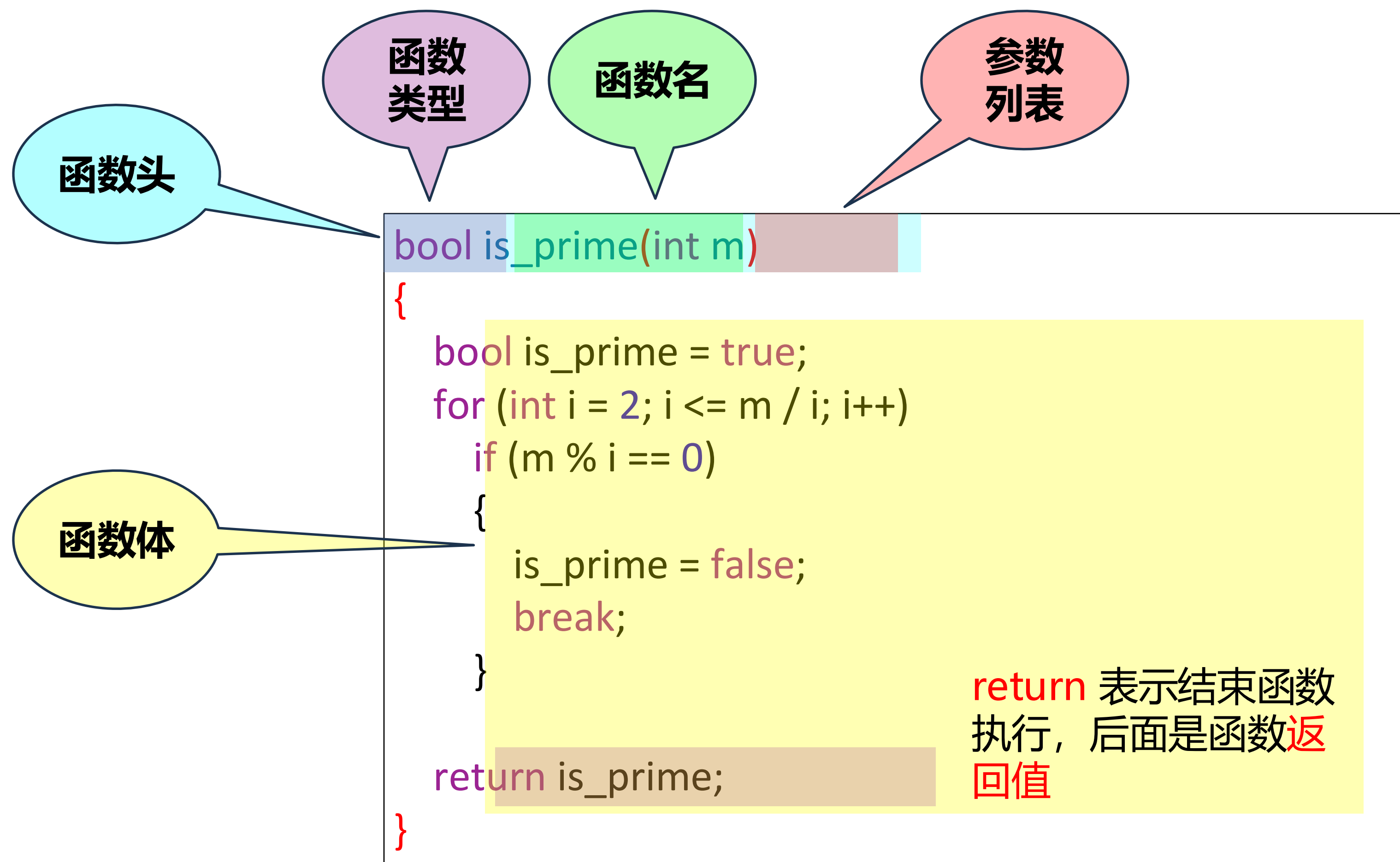
- ❑ 模块化使代码易于设计



- ❑ 函数代码具备可重用性

- ❑ 函数使代码易于理解，易于调试

函数的定义：实例1



函数的定义：实例2

无返回值
类型

无参数

```
void sum_1_100(void)
{
    int i;
    int sum = 0;
    for (i=1; i<=100; i++)
        sum += i;
    printf("1到100的和是%d\n", sum);
}
```

函数的定义

有参函数

类型名 函数名(形式参数表列)

```
{  
    函数体  
}
```

无参函数

类型名 函数名(void)

```
{  
    函数体  
}
```

1. 函数和变量一样，**必须定义类型**
2. 函数类型是**返回值**的类型
3. 函数体的**花括号**不能省略
4. 函数名必须**唯一**
5. 函数名应反映其代表的功能

函数的种类

无参数，无返回值

```
void function_name(void)
{
    ...;
}
```

有参数，无返回值

```
void function_name(parameters)
{
    ...;
}
```

无参数，有返回值

```
type function_name(void)
{
    ...;
    return value;
}
```

↗ 函数值（函数返回值）

有参数，有返回值

```
type function_name(parameters)
{
    ...;
    return value;
}
```

无返回值函数可以没有return，有返回值函数必须有

return

第六节 函数

函数的返回值

对于有返回值的函数来说：

1. return后面**表达式**的值就是函数的返回值，返回值可以赋值给变量、用于表达式、传递给函数或丢弃
2. 一个函数中可以有一个**以上**的return语句*
3. return语句中的**表达式类型**应该和**函数类型一致**，否则自动类型转换为函数类型

练习：以下程序会输出什么？

```
#include <stdio.h>
int max(float x,float y)
{ float z;
  z = x>y ? x : y;
  return z; }
```

```
int main(void)
{ float a = max(1.2, 3.3);
  printf("Max is %.1f\n", a);
  return 0; }
```

A. 3.3

B. 1.2

C. 4.0

D. 3.0

练习：以下程序会输出什么？

```
#include <stdio.h>
int max(float x,float y)
{ float z;
  z = x>y ? x : y;
  return z; }
```

```
int main(void)
{ float a = max(1.2, 3.3);
  printf("Max is %.1f\n", a);
  return 0; }
```

A. 3.3

B. 1.2

C. 4.0

D. 3.0

函数的调用

函数调用：函数名(参数值1,参数值2,...) 或 函数名()

printf("%d",i)

is_prime(i)

is_prime(7+i)

getchar()

1. 圆括号()用来标识函数，即使无参数也不可省略
2. 若有参数，则需给出参数值正确的数量、顺序与类型，参数值会按顺序初始化函数参数

sum(1,100);

```
void sum(int begin, int end)
{
    int i;
    ...;
    printf("%d到%d的和是%d\n", begin, end, sum);
}
```

形参与实参

形式参数/形参

```
void sum(int begin,int end)
{...;}
```

实际参数/实参

```
sum(1,100);
```

❑ 形式参数 (Formal Parameter)

- 函数在**定义和声明**时用的参数，需在**每个**变量前声名其类型
- 形参为**局部变量**，每次**调用函数时**才为这些变量**分配内存单元并赋值**

❑ 实际参数 (Actual Argument)

- 赋给被调函数形式参数的**值**（可为常量、变量、表达式）
- 实参与形参必须**数量相同**、**类型兼容**
- 若实、形参类型兼容但不一致，则按初始化规则**强制转换**

练习：以下程序会输出什么？

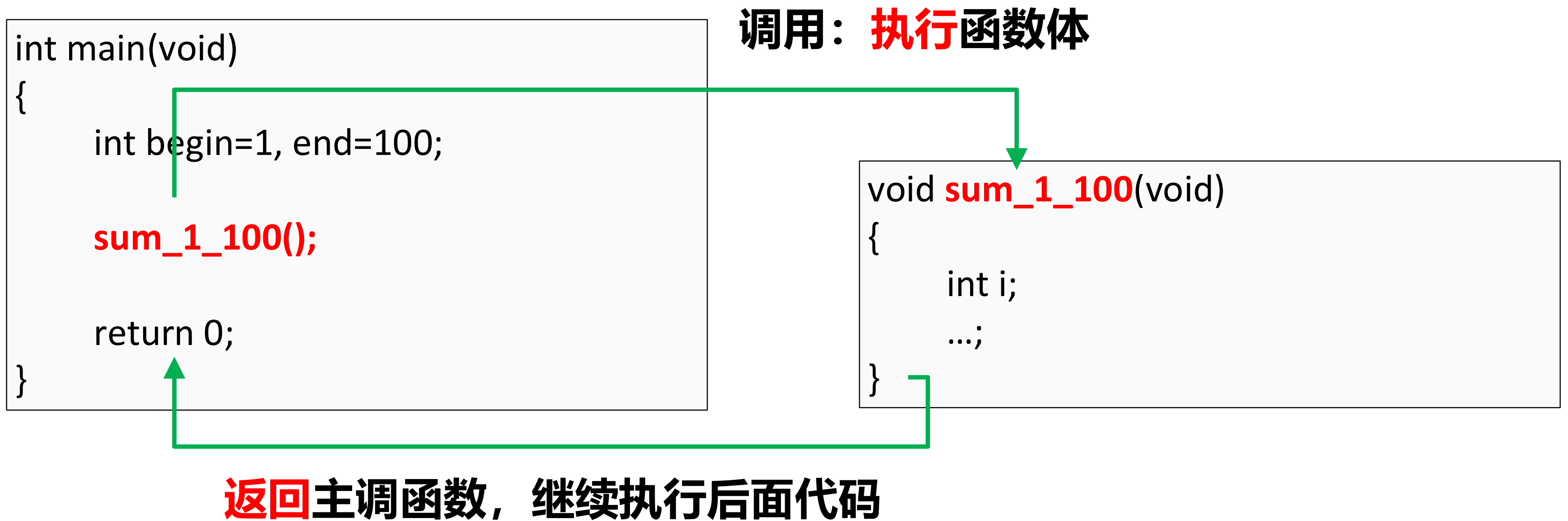
```
#include <stdio.h>
void swap(int a, int b)
{ int temp = a;
  a = b;
  b = temp; }
```

```
int main(void)
{ int a = 10, b = 20;
  swap(a,b);
  printf("a=%d b=%d\n", a, b);
  return 0; }
```

实参、形参为不同储存单元！

(回顾第三节变量作用域部分内容)

函数的调用过程：无返回值函数



函数的调用过程：有返回值函数

调用：创建形参单元，将实参**值拷贝**给形参单元，**执行函数体**

```
int main(void)
{
    int begin=1, end=100;
    int sum;

    sum = sum(1,100);

    return 0;
}
```

```
int sum(int begin, int end)
{
    int i, sum;
    ...;
    return sum;
}
```

终止函数，形参单元**释放**，**返回**sum的**值**给主调函数

函数的调用：总结

- ❑ 函数是一个初等表达式 (Primary Expression)
 - 函数返回值的类型就是表达式的类型
 - 函数遇到 return 语句或函数块执行完毕函数表达式完成求值
 - 除了返回 void 类型，必须使用 return 语句返回一个值
- ❑ 函数实参与形参
 - 实参与形参必须数量一致，类型兼容
 - 实参是传值给形参，形参为局部变量
 - 形参是传引用则实参必须是 “&左值表达式” 或 “地址” (下节课分晓)

函数的声明*

- ❑ 编译器**自上而下**分析代码
- ❑ 在看到`swap(a,b);`时需要知道swap函数是什么，才能知道对swap函数的调用是否正确
 - **参数个数**有多少？ **参数类型**分别是什么？ **返回值类型**是什么？
- ❑ 但把所有函数都定义在main函数之前会使代码读起来很费劲😞

函数的声明/函数原型*

函数类型 函数名(参数类型1 参数名1, 参数类型2 参数名2...);

- ❑ 函数类型 函数名(参数类型1 参数名1...); 被称为**函数原型**
- ❑ 目的是为了告诉编译器函数的接口信息：**名称、参数（数量与类型）和返回类型**
- ❑ 函数原型一般写在**主调函数之前**
- ❑ 函数原型可以用**不同**参数名或**省略**参数名

函数原型

```
bool is_prime(int);
```

```
int main(void) {  
    is_prime(7);  
}
```

根据原型
检查

```
bool is_prime(int m)
```

函数头

如果再省点事.....*

```
void swap(int, int);
```

vs

```
void swap();
```

为旧标准，不推荐！

头文件

- ❑ 每次逐个声明函数很麻烦，所以会按功能把一组函数声明放在一个头文件里(.h)，方便使用
- ❑ 从另一个角度看，这分离了使用者和设计者的世界，是使用者和设计者之间共同遵守的契约（接口）
 - 作为使用者，不关心函数是如何实现，我们只关心如何使用函数，有声明就足够了
 - 作为设计者，根据函数声明，编写函数实现（例如：scanf 在不同C语言编译器中实现各不相同，但使用者都可放心使用它们）

头文件

函数的声明

```
double sin(double _X);
double cos(double _X);
double tan(double _X);
double sinh(double _X);
double cosh(double _X);
double tanh(double _X);
double asin(double _X);
double acos(double _X);
double atan(double _X);
double atan2(double _Y,double _X);
double exp(double _X);
double log(double _X);
double log10(double _X);
double pow(double _X,double _Y);
double sqrt(double _X);
double ceil(double _X);
double floor(double _X);
```

一目了然，一眼知道如何调用 😊

函数的定义

```
__mingw_ovr
__attribute__((__format__(gnu_scanf, 1, 2))) __MINGW_ATTRIB_NONNULL(1)
int scanf(const char *__format, ...)
{
    int __retval;
    __builtin_va_list __local_argv; __builtin_va_start( __local_argv, __format );
    __retval = __mingw_vfscanf( stdin, __format, __local_argv );
    __builtin_va_end( __local_argv );
    return __retval;
}

__mingw_ovr
__attribute__((__format__(gnu_scanf, 2, 3))) __MINGW_ATTRIB_NONNULL(2)
int fscanf(FILE *__stream, const char *__format, ...)
{
    int __retval;
    __builtin_va_list __local_argv; __builtin_va_start( __local_argv, __format );
    __retval = __mingw_vfscanf( __stream, __format, __local_argv );
    __builtin_va_end( __local_argv );
    return __retval;
}
```

需要在万千代码中辛苦找到函数接口 😭

C程序与函数

- ❑ C程序由若干函数构成（**必须包含 main** 函数作为程序入口）
- ❑ 函数允许我们**模块化程序**

```
#include <stdio.h>
#include <stdbool.h>

bool is_prime(int);

int main(void)
{
    int n;
    scanf("%d", &n);

    for(int p=2; p<=n; p++){
        if(is_prime(p) && is_prime(n-p)){
            printf("%d = %d + %d", n, p, n-p);
            return 0;
        }
    }
}
```

```
bool is_prime(int m)
{
    bool is_prime = true;
    for (int i = 2; i <= m / i; i++)
        if (m % i == 0)
        {
            is_prime = false;
            break;
        }
    return is_prime;
}
```

常用数学函数

- 在标头 `stdlib.h` 中定义
 - `int abs( int n );`
- 定义于头文件 `math.h`
 - `double fabs( double arg );`
 - `double exp( double arg );`
 - `double log( double arg );`
 - `double log10( double arg );`
 - `double pow( double base, double exponent );`
 - `double sqrt( double arg );`
 - `double sin( double arg );`
 - `cos, tan, asin, acos, atan`
 - `double ceil( double arg );`
 - `double floor( double arg );`

要求掌握!

练习：电报加密

输入一行电报文字，将字母变成其下一字母（如 'a' 变成 'b' 'z' 变成 'a'，其它字符不变）。

样例输入：a b

样例输出：b c

请打开 6_01_encipher.c 结合代码注释完成程序

主要内容

1. 函数的概念

2. 函数的定义、调用与声明

3. 递归

递归



递归跟套娃很像！

递归示例：阶乘

- n 的阶乘就是所有小于等于 n 的正整数的积：

$$n! = n * \underbrace{(n-1) * (n-2) * \cdots * 1}_{(n-1)!}$$

- n 的阶乘还可以这样定义：

$$n! = \begin{cases} 1 & \text{if } n = 1 \\ n * (n-1)! & \text{if } n > 1 \end{cases} \rightarrow \text{fac}(n) = n * \text{fac}(n-1);$$

- 第一种叫做迭代定义；第二种就叫做**递归定义**，包含
 1. **基本情况**（Bottom Cases），基本的事实（终止条件）
 2. **递推关系**（Recurrence Relation），通过递推**拆解**递归到基本情况，给出问题解
- 在程序设计中，**递归**是一种问题求解的方法：**函数自己调用自己**

递归示例：阶乘

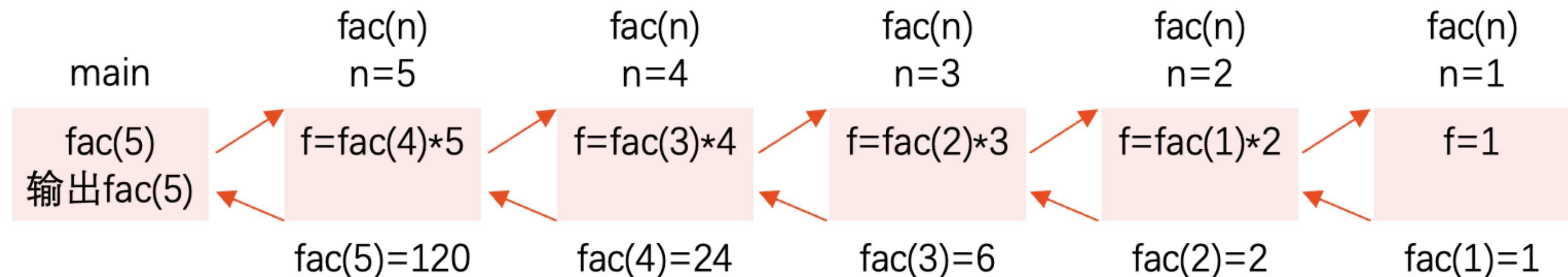
```
int fac_loop(int n) {
    int out = 1;
    for (int i=n; i>0; i--)
        out *= i;
    return out;
}

int fac(int n) {
    if (n <= 1)
        return 1;
    else
        return n * fac(n - 1);
}
```

```
/*Facterial*/
#include<stdio.h>

int fac_loop(int);
int fac(int);

int main(void) {
    /*请使用 -1,0,1,2,5 测试程序*/
    int n = 5;
    printf("loop out %d\n", fac_loop(n));
    printf("recursive out %d\n", fac(n));
}
```



主要内容

1. 函数的概念

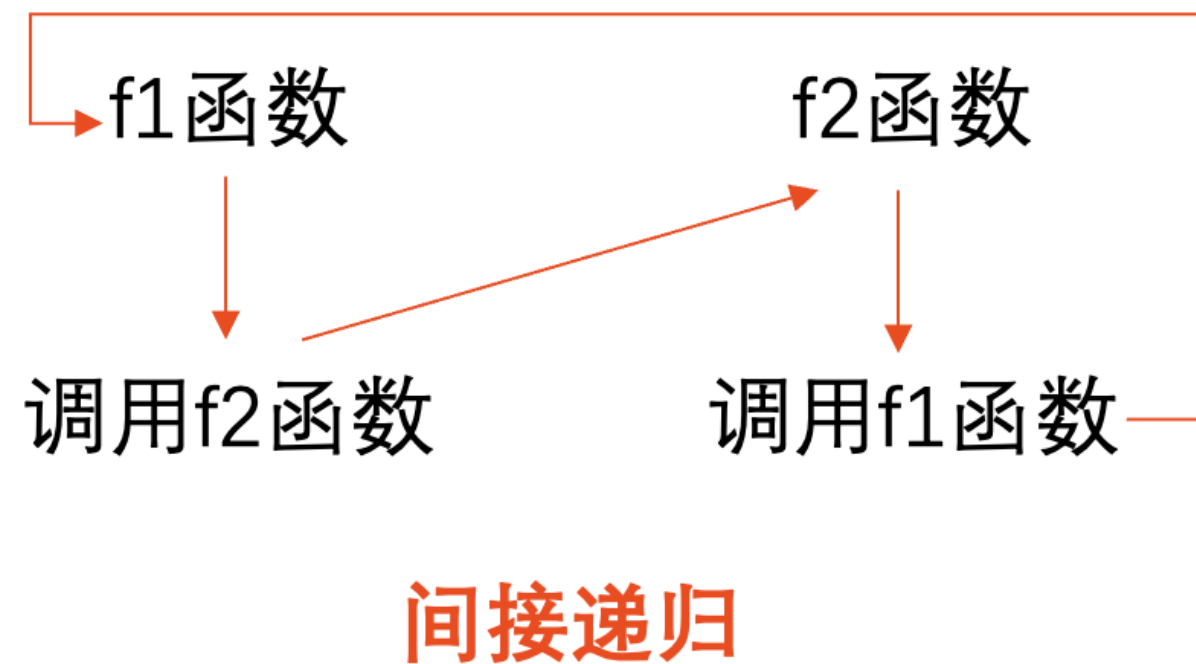
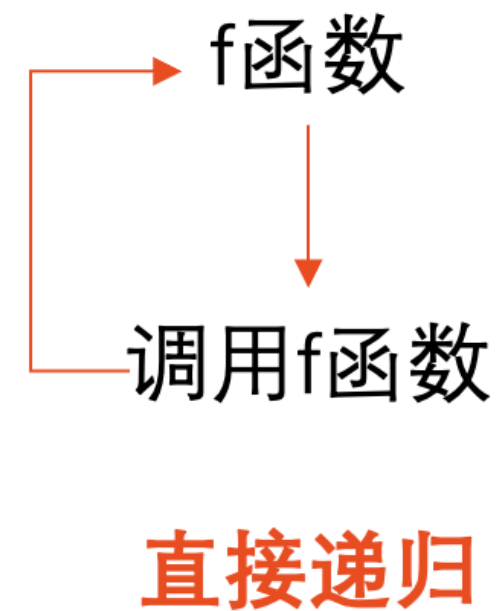
2. 函数的定义、调用与声明

3. 递归

4. 怎么用递归

递归调用

- 在调用一个函数的过程中又出现**直接或间接地调用该函数本身**，称为函数的**递归调用**



- 程序中**不应出现无终止的递归调用**，而只应出现有限次数的、有终止的递归调用
- 这可以用**if语句来控制**，只有在某一条件成立时才继续执行递归调用；否则就不再继续

递归练习1：整数求和

用递归计算 $1+2+3+\dots+n$ 。

```
int sum(int n) {
    if (n != 0)
        // sum() function calls itself
        return n + sum(n-1);
    else
        return n;
}

int main() {
    int number, result;

    printf("Enter a positive integer: ");
    scanf("%d", &number);

    result = sum(number);

    printf("sum = %d", result);
    return 0;
}
```

递归练习2：青蛙跳台阶

题目描述

一只青蛙一次可以跳上 1 级台阶，也可以跳上 2 级。求该青蛙跳上一个 n 级的台阶总共有多少种跳法。

解题思路

$$F(n) = \begin{cases} 1 & \text{when } n = 1 \\ 2 & \text{when } n = 2 \\ F(n-1) + F(n-2) & \text{when } n > 2 \end{cases}$$

递归可能存在的问题？ **消耗内存空间！**

尾递归

- ❑ 尾递归：递归调用处于函数末尾
- ❑ 尾递归是最简单的递归形式，可用循环替代

```
/* 计算Fibonacci数：递归实现 */  
long long f(int n){  
    if (n<=2) return 1;  
    return f(n-1) + f(n-2);  
}
```

```
/* 计算Fibonacci数：循环实现 */  
long long f(int n){  
    if (n<=2) return 1;  
    long long fn, f1=1, f2=1;  
    for (int i=1; i<=n-2; i++){  
        fn = f2 + f1;  
        f1 = f2;  
        f2 = fn;  
    }  
    return fn;  
}
```

递归练习3：最大公约数

□ 计算方法：**辗转相除法**

□ 算法定义：

$$\gcd(a, b) = \begin{cases} a & \text{if } b \% a == 0 \\ \gcd(b \% a, a) & \text{if } b \% a \neq 0 \end{cases}$$

□ 例如：

$$\gcd(98, 63) = \gcd(63, 98) = \gcd(35, 63) = \gcd(28, 35) = \gcd(7, 28) = 7$$

□ 这么巧妙的方法是谁想出来的？

■ 出于**欧几里得**的《几何原本》，它是许多整数问题的基础函数，如计算最小公倍数等

递归练习4：全排列

7-3 输出全排列 分数 10

[查看题目详情](#) [全屏浏览](#) [切换布局](#)

请编写程序输出前 n 个正整数的全排列 ($n < 10$)，并通过9个测试用例（即 n 从1到9）观察 n 逐步增大时程序的运行时间。

输入格式：

输入给出正整数 n (< 10)。

输出格式：

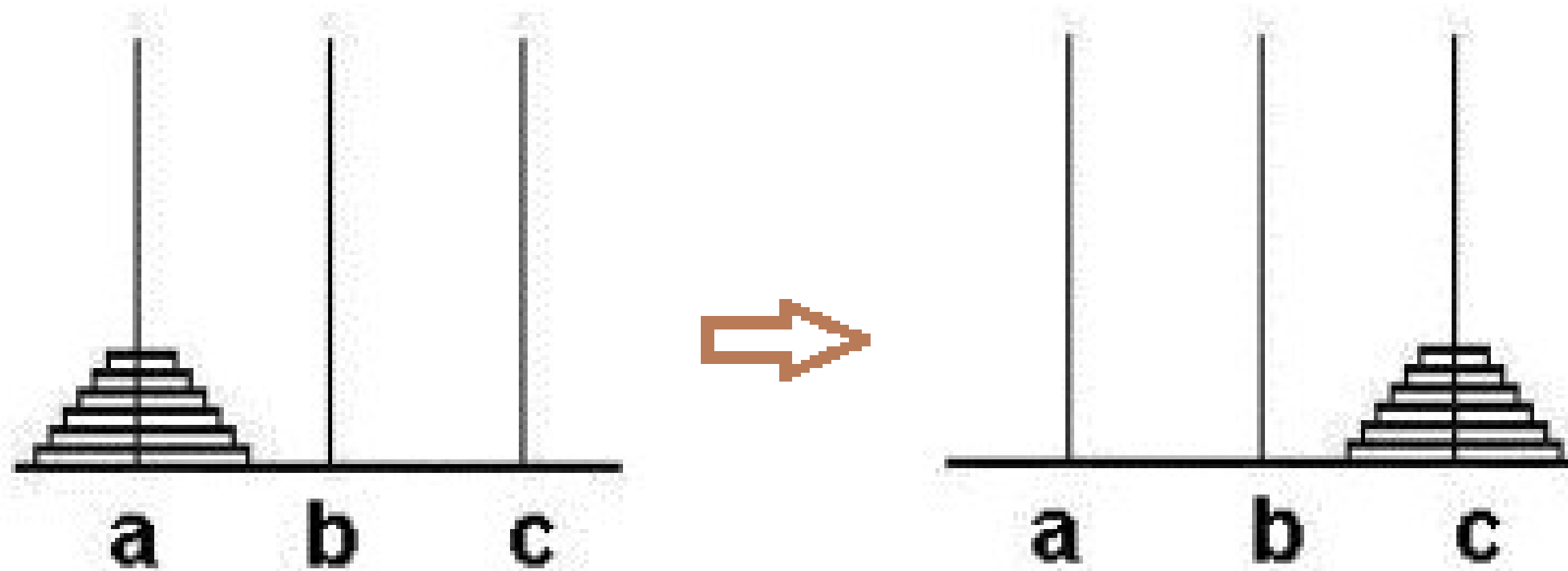
输出1到 n 的全排列。每种排列占一行，数字间无空格。排列的输出顺序为字典序，即序列 $a_1, a_2, \dots, a_n$ 排在序列 $b_1, b_2, \dots, b_n$ 之前，如果存在 k 使得 $a_1 = b_1, \dots, a_k = b_k$ 并且 $a_{k+1} < b_{k+1}$ 。

写程序技巧

1. 从实例出发，**找思路**
2. 第 1 版伪代码，第 2 版伪代码.....
3. **迭代**直到看起来不错
4. **实现**代码
5. **Debug**
6. 完成 🎉

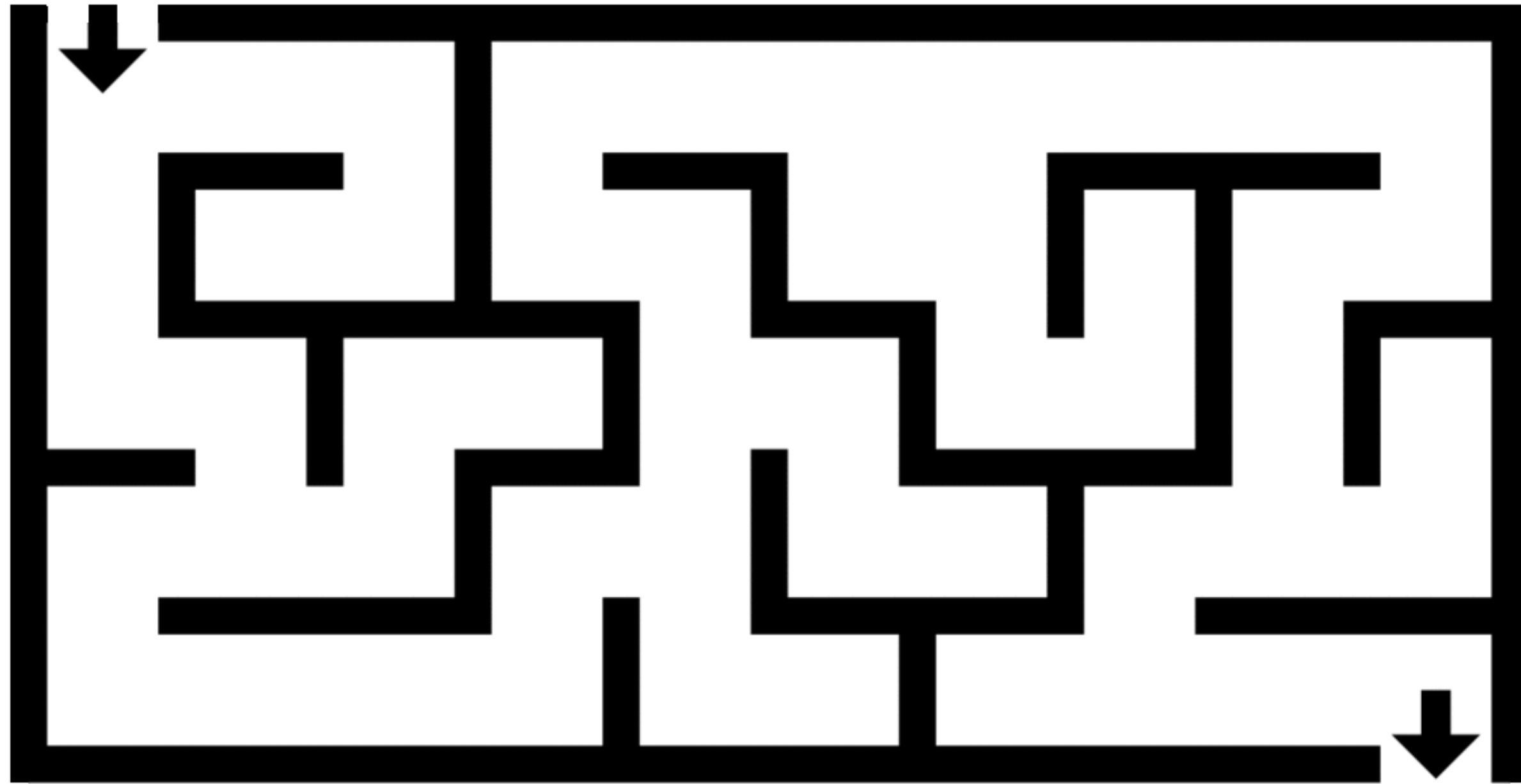
递归练习5：汉诺塔 (Tower of Hanoi)

- 源于印度一个古老传说：大梵天创造世界的时候做了三根金刚石柱子，在一根柱子上从下往上按照大小顺序摞着64片黄金圆盘。大梵天命令婆罗门把圆盘从下面开始**按大小顺序重新摆放在另一根柱子上**。并且规定，任何时候，在**小圆盘上不能放大圆盘**，且在三根柱子之间**一次只能移动一个圆盘**。问应该如何操作？



递归：探索迷宫

- ❑ 递归的概念至关重要，它经常用在数据结构和算法中，比如图的搜索与回溯



- ❑ 递归使程序代码优雅简洁，但因调用函数的缘故，递归通常要慢得多

头文件

- ❑ 为了确保函数**开发者**和**使用者**使用相同的声明和定义，C语言使用头文件（拓展名为 .h）书写这些申明与定义
- ❑ 使用 **#include** 编译预处理指令将头文件插入即将编译的源代码中

1. 将 main.c 修改为:

```
#include <stdio.h>
#include "mymath.h"

int main(void) {
    ...
}
```

fac.h 头文件

```
#pragma once

/* get the square */
double square(double);
```

mymath.c 原文件

```
#include <stdio.h>

double square(double a){
    return a*a;
}
```

2. 将 **#include "mymath.h"** 添加入
main.c

3. 添加 mymath.c 源文件，并重新编译运行

头文件

- 头文件通常包含：
 - 共享的函数声明
 - 全局变量声明与初始化
 - 头文件中函数与变量不能用 `static` 修饰
 - 定义的预处理符号或宏
- 头文件通过 `#include` 编译预处理指令添加到源代码文件中
 - 例如：源代码中需要使用内置库的求绝对值这个函数
 - 第一种方法，使用标准的头文件：`#include <math.h>`
 - 第二种方法，自己给出声明函数：`double fabs (double);`
- 注意：
 - 自定义的头文件使用`"`号；
 - 头文件第一行使用 `#pragma once`，指令确保编译仅展开一次

纸上得来终觉浅！

邮箱: yuesh5@mail.sysu.edu.cn